

JAVA FULL STACK DEVELOPER VIRTUAL INTERNSHIP

An Internship Report submitted
in partial fulfillment of the
Requirements for the award of the degree of

Bachelor of Technology

In

Artificial Intelligence & Machine Learning

By

SHAIK SANA

Reg No: 22H71A6151

OFFERED BY

AICTE-EDUSKILLS



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

DVR & Dr. HS

MIC College of Technology

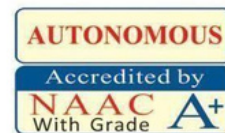
Autonomous

Kanchikacherla– 521180, NTR Dist, Andhra Pradesh

JAN - MAR 2026



DVR & Dr. HS
MIC College of Technology
ISO 9001:2015 Certified Institute
(Approved by AICTE & Permanently Affiliated to JNTUK, Kakinada)
Kanchikacherla - 521180, NTR Dist., A.P., India
Phones : 08678 - 273535 / 94914 57799 / 73826 16824
E mail: office@mictech.ac.in, Website: www.mictech.edu.in



CERTIFICATE

This is to certify that the Internship Report entitled “***JAVA FULL STACK DEVELOPER Virtual Internship***” submitted by ***SHAIK SANA (22H71A6151)***, to the DVR & Dr. HS MIC College of Technology in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in ***Artificial Intelligence & Machine Learning*** is a Bonafide record of work.

Internship Coordinator

Head of the Department

Examiner1

Examiner2

Principal



शिक्षा मंत्रालय
MINISTRY OF
EDUCATION



अखिल भारतीय तकनीकी शिक्षा परिषद्
All India Council for Technical Education



EduSkills®
Nation Building Through Skills



Certificate of Virtual Internship

This is to certify that

Shaik Sana

DVR & Dr. HS MIC College of Technology

has successfully completed the 10-weeks

Java Full Stack Developer Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Supported by



EduSkills
ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education



Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 430c927072170a4d0370

Student ID: STU6625388b303711713715339



GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose constant guidance and engagement crown all the efforts with success. I thank our college management and respected ***Sri D. PANDURANGA RAO, CEO*** for providing us the necessary infrastructure to carry out the Internship.

I express my sincere thanks to ***Dr. T. Vamsee Kiran, Principal*** who has been a great source of inspiration and motivation for the internship program.

I profoundly thank ***Dr. G Sai Chaitanya Kumar, Head, Department of AI*** for permitting me to carry out the internship.

I am thankful to the ***AICTE and Android Developer*** for enabling me an opportunity to carry out the internship in such a prestigious organization.

I am thankful to our Internship Coordinator ***Dr. A. Kalyan Kumar***, Associate Professor, Department of AI for their internal support and professionalism who helped us in completing the internship on time.

I take this opportunity to express our thanks to one and all who directly or indirectly helped me in bringing this effort to present form.

Finally, my special thanks go to my family for their continuous support and help throughout and for their continual support and encouragement for the completion of the Internship on time.

Abstract

This report summarizes my internship experience as a Java Full Stack Developer, where I contributed to the development and enhancement of web applications using Java, Spring Boot, and various front-end technologies. Throughout the internship, I was involved in the entire software development lifecycle, from requirements gathering and system design to implementation, testing, and deployment.

I gained hands-on experience with RESTful API development, database management using MySQL, and front-end frameworks such as Angular. Collaborating closely with cross-functional teams, I learned best practices in agile methodologies and version control using Git. My contributions included optimizing application performance, improving user interface responsiveness, and implementing new features based on client feedback.

This report reflects on the technical skills acquired, challenges faced, and the overall impact of my work on project outcomes. It also highlights the importance of continuous learning and adaptability in the rapidly evolving tech landscape.

INDEX

SNO	CONTENTS	PAGENO
1	Title Page	1
2	Certificate	2-3
3	Acknowledgement	4
4	Abstract	5
5	Index page	6-7
6	List of figures	8
7	1.JAVA	9
	1.1-Introduction to java	9
	1.2-Features of java	9
	1.3-Basic structure of java	9
8	2.HTML	
	2.1-Introduction to HTML	10
	2.2-Basic structure of HTML	10
	2.3-Common Elements &Multimedia tags	11
	2.4 -Forms	12
	2.5-Semantic HTML	12
	2.6- HTML Attributes &Tags	12
	2.7-HTML-5 Features	13
	2.8-Best Practices	13
	2.9-Importance of HTML	13
	2.10-Conclusion	13
10	3.CSS	14
	3.1 Introduction to CSS	14
	3.2-Basic syntax of CSS	14
	3.3-Types of CSS	14
	3.4-Selectors	15
	3.5-Layout Techniques	16
	3.6 Responsive Design	17
	3.7 CSS Frames Works	18
	3.8 Best Practices	18

	3.9 Conclusion	18
11	4.JAVA SCRIPT	19
	4.1 Popular use cases	19
	4.2 Resources for learning	20
	4.3 Ecosystem	20
12	5.Frame works	
	5.1 Front End Frame Works	21
	5.2 Back End Frame Works	22
	5.3 Full Stack Frame works	22
	5.4 Testing Frameworks	23
13	6.Angular	24
	6.1 Key Features	24
	6.2 Common Use Cases	25
14	7.SQL (Structured Query Language)	26
	7.1 What is Data Base?	26
	7.2 Why To Learn SQL?	27
15	8.AJAX	28
	8.1 Introduction	28
	8.2 How AJAX Works	28
16	9.Servlets	29
17	10.Hibernate	30-31
18	11.Spring Boot	32
19	12.Conclusion	34
20	13.Case Study	35-36

LIST OF FIGURES

FIG-NO	FIGURE NAME	PAGE NO
Fig 2.1	Forms	12
Fig 3.1	Internal css	15
Fig 3.2	External css	15
Fig 3.3	Flex layout	16
Fig 3.4	Grid layout	17
Fig 3.5	Responsive design	17
Fig 7.1	Data base	27
Fig 9.1	Server	29
Fig 10.1	Orm tool	30
Fig 11.1	Spring boot	32

1- Introduction to Java

Java Full Stack:

A full-stack developer is a person who can develop application's backend and frontend. Java full-stack is basically a term used for a web developer that uses Java to develop the entire technology stack is referred to as Java full stack developer.

Introduction to Java:

Java is a popular programming language, created in 1995.

- It is owned by Oracle, and more than 3 billion devices run Java.
- Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.).
- It is easy to learn and simple to use.
- It is open source and free.
- It is secure, fast and powerful.
- Java is an object-oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs.
- It is used for:
 - Mobile applications
 - Web applications
 - Servers

Syntax: public class Main

```
{  
  
    public static void main (String[] args)  
  
    {  
  
        System.out.println("Hello World");  
  
    }  
  
}
```

2.HTML

HTML is an acronym which stands for Hyper Text Markup Language which is used for creating web pages and web applications. Let's see what is meant by Hypertext Markup Language, and Web page.

Hypertext: Hypertext simply means "Text within Text." A text has a link within it, is a hypertext. Whenever you click on a link which brings you to a new web page, you have clicked on a hypertext. Hypertext is a way to link two or more web pages (HTML documents) with each other.

Markup language: A markup language is a computer language that is used to apply layout and formatting conventions to a text document. Markup language makes text more interactive and dynamic. It can turn text into images, tables, links, etc.

Web Page: A web page is a document which is commonly written in HTML and translated by a web browser. A web page can be identified by entering an URL. A Web page can be of the static or dynamic type. With the help of HTML only, we can create static web pages.

Hence, HTML is a markup language which is used for creating attractive web pages with the help of styling, and which looks in a nice format on a web browser. An HTML document is made of many HTML tags and each HTML tag contains different content.

1. Basic structure of HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Document Title</title>
</head>
<body>
  <h1>Main Heading</h1>
  <p>This is a paragraph of text. </p>
</body>
</html>
```

Components:

- `<!DOCTYPE html>`: Declaration defining the document type and version of HTML.
- `<html>`: Root element of the document.
- `<head>`: Contains metadata, links to stylesheets, and scripts.
- `<body>`: Contains the visible content of the webpage.

2. Common HTML

Elements Text

Elements

- **Headings:** Defined by `<h1>` to `<h6>`, with `<h1>` being the most important.
- **Paragraphs:** `<p>` defines a block of text.
- **Links:** `<a href="URL">Link text</a>` creates hyperlinks.

Lists:

Ordered List: `<ol>` for numbered items.

Unordered List: `<ul>` for bullet points.

List Item: `<li>` used within both `<ol>` and `<ul>`.

Multimedia Element

Images: `` to embed images.

Audio: `<audio controls><source src="audio.mp3" type="audio/mpeg">Your browser does not support the audio element.</audio>`.

Video: `<video controls><source src="video.mp4" type="video/mp4">Your browser does not support the video tag.</video>`.

3. FORMS:

Forms are crucial for user interaction:

```
html

<form action="/submit" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>
  <input type="submit" value="Submit">
</form>
```

Fig 2-forms

- <form>: Encloses form elements.
- <input>: Various types (text, password, checkbox, radio, etc.).
- <label>: Associates a label with a form control.

4. Semantic HTML

Using semantic HTML elements enhances accessibility and SEO. Here are some semantic tags:

- <header>: Defines introductory content or navigational links.
- <footer>: Represents the footer for a section or page.
- <article>: Self-contained content that could stand alone.
- <section>: Thematic grouping of content.

5. HTML Attributes

Attributes provide additional information about elements:

- **Global Attributes:** Commonly used across all HTML elements(e.g., class, id, style, title).
- **Specific Attributes:**

Unique to certain elements, like src for or href for <a>.

6. HTML5 Features

HTML5 introduced significant improvements:

- **New Input Types:** Such as email, date, color, improving forms and validation.
- **Offline Capabilities:** Using the application Cache API.
- **APIs:** Geolocation, Web Storage (local and session storage), and Canvas for graphics.

7. Best Practices

- **Use Semantic Elements:** Improve accessibility and SEO.
- **Validate HTML:** Use tools like the W3C Validator to ensure code quality.
- **Optimize for Accessibility:** Use alt attributes for images, proper heading levels, and ARIA roles when necessary.
- **Keep It Clean:** Organize code for readability—indentation, comments, and consistent naming conventions help maintainability.

8. Importance of HTML:

- **Structure:** HTML provides a framework for organizing content on the web.
- **Accessibility:** Proper use of HTML elements improves accessibility for users with disabilities.
- **SEO:** A well-structured HTML document can improve search engine rankings.

9. Conclusion

HTML is the foundation of web development, allowing you to create structured and accessible web content. Understanding its elements, attributes, and best practices is essential for building modern web applications.

3-CSS

Introduction:

CSS (Cascading Style Sheets) is a stylesheet language used to describe the presentation of a document written in HTML or XML. CSS controls the layout, colors, fonts, and overall visual appearance of web pages.

1. Basic Syntax of CSS:

CSS consists of selectors and declaration blocks.

Example:

```
``css
selecto
r {
    property: value;
}
``
```

- **Selector**: Targets the HTML element(s) you want to style (e.g., `h1``, `.class``, `#id``).
- **Property**: The CSS property you want to change (e.g., `color``, `font-size``, `margin``).
- **Value**: The value assigned to the property (e.g., `blue``, `16px``, `10px``).

2. Types of CSS:

There are three main ways to apply CSS to HTML:

1. Inline CSS:

Applied directly to an HTML element using the `style`` attribute.``html<h1 style="color: blue;">Hello, World! </h1>

```
html

<head>
  <style>
    body {
      background-color: lightgrey;
    }
  </style>
</head>
```

Fig 3.1-internal css

External CSS:

Written in a separate `.css` file and linked to the HTML document using the `` tag.

```
html

<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

Fig 3.2-external css

2. Selectors: CSS offers various selectors to apply styles:

- Element Selector: Targets specific HTML elements (e.g., `p`).
- Class Selector: Targets elements with a specific class (e.g., `.class-name`).
- ID Selector: Targets a unique element with a specific ID (e.g., `#id-name`).
- Attribute Selector: Targets elements based on an attribute (e.g., `[type="text"]`).
- Pseudo-classes: Style elements based on their state (e.g., `:hover`, `:first-child`).
- Pseudo-elements: Style specific parts of an element (e.g., `::before`, `::after`).

3. Layout Techniques:

CSS provides several layout models:

1. Flexbox:

A one-dimensional layout model for arranging items in rows or columns.

```
CSS

.container {
  display: flex;
  justify-content: center; /* Align items horizontally */
  align-items: center;     /* Align items vertically */
}
```

Fig 3.3 flex layout

2. Grid: A two-dimensional layout model that allows for complex layouts.

```
CSS

.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
  grid-gap: 10px; /* Space between grid items */
}
```

Fig 3.4 Grid layout

5. CSS Properties:

Some common CSS properties include:

- **Color and Background:** `color`, `background-color`, `background-image`.

Text: `font-family`, `font-size`, `text-align`, `line-height`.

Box Model: `margin`, `padding`, `border`.

Layout: `display`, `flex`, `grid`.

Transitions and Animations: `transition`, `animation`.

6. Responsive Design:

Creating web designs that adapt to various screen sizes is essential.

Use:

Media Queries: Apply different styles based on device characteristics.

A code editor window with a dark background and a title bar that says "CSS". The code inside is a media query that sets the background color of the body to lightblue for screens with a maximum width of 600px.

```
@media (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

Fig 3.5-Responsive design

Viewport Units: `vw`, `vh`, `vmin`, `vmax` for responsive sizing.

6. CSS Frameworks:

Bootstrap: A responsive framework with pre-designed components.

- **Tailwind CSS:** A utility-first framework for rapid UI development.
- **Foundation:** A responsive front-end framework similar to Bootstrap.

7. Best Practices:

- **Organize Your CSS:** Use comments and logical grouping for readability.
- **Minimize CSS File Size:** Remove unused styles and use minification.
- **Use Classes Over IDs:** Classes are reusable and help avoid specificity issues.
- **consider Performance:** Avoid excessive selectors and complexity to enhance rendering speed.

Conclusion

CSS is essential for web design, allowing developers to create visually appealing and responsive web pages. Understanding its syntax, properties, and best practices is key to mastering web development.

4-JAVASCRIPT

JavaScript (js) is a light-weight object-oriented programming language which is used by several websites for scripting the webpages. It is an interpreted, full-fledged programming language that enables dynamic interactivity on websites when applied to an HTML document. It was introduced in the year 1995 for adding programs to the webpages in the Netscape Navigator browser. Since then, it has been adopted by all other graphical web browsers. With JavaScript, users can build modern web applications to interact directly without reloading the page every time. The traditional website uses js to provide several forms of interactivity and simplicity.

- Although, JavaScript has no connectivity with Java programming language. The name was suggested and provided in the times when Java was gaining popularity in the market. In addition to web browsers, databases such as CouchDB and MongoDB uses JavaScript as their scripting and query language. Application of JavaScript
- JavaScript is used to create interactive websites. It is mainly used for:
- Client-side validation,
- Dynamic drop-down menus,
- Displaying pop-up windows and dialog boxes (like an alert dialog box, confirm , dialog box and prompt dialog box),
- Displaying clocks etc.

Popular Use Cases:

- **Web Development:** Building interactive web applications using frameworks like Angular, or Vue.js.
- **Server-Side Development:** Using Node.js to build back-end services.
- **Mobile App Development:** Frameworks like React Native allow you to build mobile applications.
- **Game Development:** Libraries like Phaser for building browser games.
- **Automation:** Scripts for automating tasks in web browsers or servers.

Resources for Learning:

- **DN Web Docs:** Comprehensive documentation and tutorials.
- **JavaScript.info:** A detailed guide covering modern JavaScript.
- **Free Code Camp:** Offers interactive coding lessons.
- **Code academy:** Provides hands-on JavaScript courses.

Ecosystem:

JavaScript has a rich ecosystem with numerous frameworks and libraries, such as:

- **React:** A library for building user interfaces.
- **Angular:** A framework for building single-page applications.
- **Vue.js:** A progressive framework for building user interfaces.

Conclusion:

JavaScript is essential for modern web development, enabling developers to create dynamic, interactive, and user-friendly web applications. Its versatility allows it to be used in a wide range of environments beyond the web browser. If you want to know more about specific features or concepts, just let me know!

5-FARME WORKS

JavaScript frameworks are pre-written JavaScript libraries that provide a structured way to build applications. They help streamline development by offering reusable components and tools for common tasks. Here are some of the most popular JavaScript frameworks

Front-End Frameworks:

1. React:

- **Description:** A library for building user interfaces, developed by Facebook. It uses a component-based architecture and is known for its virtual DOM for efficient rendering.

- **Use Cases:** Single-page applications (SPAs), complex UIs.

2. Angular:

- **Description:** A robust framework developed by Google for building dynamic web applications. It uses TypeScript and offers features like two-way data binding and dependency injection.

- **Use Cases:** Enterprise applications, SPAs.

3. Vue.js:

- **Description:** A progressive framework that is easy to integrate into projects. It provides reactive data binding and a component-based architecture.

- **Use Cases:** Small to medium applications, single-page applications.

4. Svelte:

- **Description:** A relatively new framework that shifts much of the work to compile time, resulting in highly optimized, lightweight applications.

- **Use Cases:** SPAs, high-performance applications.

Back-End Frameworks:

1. Node.js:

- **Description:** A runtime environment that allows JavaScript to be run on the server side. It uses an event-driven, non-blocking I/O model.
- **Use Cases:** RESTful APIs, real-time applications (like chat apps).

2. Express.js:

- **Description:** A minimal and flexible Node.js web application framework that provides robust features for web and mobile applications.
- **Use Cases:** Building web applications and APIs.

3. Nest JS:

- **Description:** A framework for building efficient, scalable Node.js server-side applications. It uses TypeScript and is heavily inspired by Angular.
- **Use Cases:** Enterprise-grade applications, microservices.

Full-Stack Frameworks

1. Meteor:

- **Description:** A full-stack platform for building web and mobile applications in pure JavaScript. It integrates front-end and back-end development seamlessly.
- **Use Cases:** Real-time applications, rapid prototyping.

2. Next.js:

- **Description:** A React framework for server-side rendering and generating static websites. It provides features like routing and API endpoints.
- **Use Cases:** SEO-friendly applications, static sites.

3. Nuxt.js

- **Description:** A framework built on top of Vue.js for creating server-rendered applications or generating static websites.
- **Use Cases:** SEO-optimized applications, SPAs.

Testing Frameworks:

1. Jest:

- **Description:** A JavaScript testing framework maintained by Facebook, commonly used for testing React applications.
- **Use Cases:** Unit testing, integration testing.

2. Mocha:

- **Description:** A flexible testing framework for Node.js and the browser that supports various assertion libraries.
- **Use Cases:** Unit testing, API testing.

3. Cypress:

- **Description:** A testing framework for end-to-end testing of web applications.
- **Use Cases:** Functional testing, UI testing.

Conclusion:

Choosing the right framework depends on the project requirements, team familiarity, and specific use cases. Each framework has its strengths, and many developers use a combination of frameworks and libraries to achieve their goals. If you want more detailed information on a specific framework or its features, just let me know!

6-Angular

Angular is a robust, open-source web application framework developed and maintained by Google. It is designed to facilitate the development of single-page applications (SPAs) using a component-based architecture. Here's an overview of Angular:

Key Features

1. Component-Based Architecture: Angular applications are built using components, which encapsulate the HTML, CSS, and JavaScript necessary for a particular part of the user interface.

2. TypeScript: Angular is built using TypeScript, a superset of JavaScript that adds static typing. This enhances code quality and helps catch errors during development.

3. Two-Way Data Binding: Angular allows for automatic synchronization of data between the model and the view, simplifying the management of data flow in applications.

4. Dependency Injection: Angular uses a built-in dependency injection system, which makes it easier to manage services and improve modularity and testability.

5. Directives: Directives are special markers in the DOM that extend HTML's capabilities by adding custom behavior or modifying the DOM.

6. Routing: Angular includes a powerful routing module that allows developers to create a single-page application experience with multiple views and deep linking.

7. Reactive Programming: Angular supports reactive programming with RxJS, allowing for more manageable asynchronous data streams.

8. Comprehensive Tooling: Angular CLI (Command Line Interface) provides powerful tools for scaffolding, building, and deploying applications efficiently.

- Common Use Cases:

- **Single-Page Applications (SPAs):** Angular is ideal for creating SPAs where the content dynamically updates without requiring a full page reload.
- **Enterprise Applications:** Its structure and scalability make it suitable for large-scale applications that require maintainability and robust architecture.
- **Progressive Web Applications (PWAs):** Angular can be used to build PWAs that offer a native-like experience on the web.

Getting Started:

To start developing with Angular, you'll typically:

1. **Install Angular CLI:** Use npm to install the Angular Command Line Interface.

```
```bash
npm install -g @angular/cli
```
```

2. **Create a New Project:** Use the CLI to generate a new Angular project.

```
```bash
ng new my-angular-app
```
```

3. **Serve the Application:** Navigate to the project directory and start a local development server.

```
```bash
cd my-
angular-app
serve
```

## Conclusion:

Angular is a powerful framework for building modern web applications, with a strong emphasis on maintainability, testability, and performance. It's widely used in the industry, particularly for complex applications that require a solid structure and scalability. If you have specific questions or want to dive deeper into any aspect of Angular, let me know!

## SQL (Structured query language)

SQL is a standard database language used to access and manipulate data in databases. SQL stands for Structured Query Language. SQL was developed by IBM Computer Scientists in the 1970s. By executing queries SQL can create, update, delete, and retrieve data in databases like MySQL, Oracle, PostgreSQL, etc. Overall SQL is a query language that communicates with databases.

### **What is a Database?**

Data is the new fuel of this world but only data is unorganized information, so to organize that data we make a database. A database is the organized collection of structured data which is usually controlled by a database management system (DBMS). Databases help us with easily storing, accessing, and manipulating data held on a computer.

### **Why to learn SQL?**

Without a lot of coding knowledge, we can manage a database with SQL. SQL works with database systems from Oracle, IBM, Microsoft, etc.

Simple and easy to learn.

SQL is ANSI and ISO standard language for database manipulations.

SQL retrieves large amounts of data very fast. Applications of SQL In data-driven Industries where managing data bases is very important in regular.

### **Here are some important SQL applications.**

To support client/server architecture, software engineers use SQL to establish the connection between back-end and front-end

- SQL can also be used in the 3-tier architecture of a client, an application server, and a database.
- SQL is used as a Data Definition Language (DDL) in which we can independently create a database, define the structure, use it, and discard it when its work is done.
- SQL is used as a Data Manipulation Language (DML) in which we can enter data, modify data, extracting data.
- SQL is used as a Data Control Language (DCL) it specifies how we can protect our database against corruption and misuse.

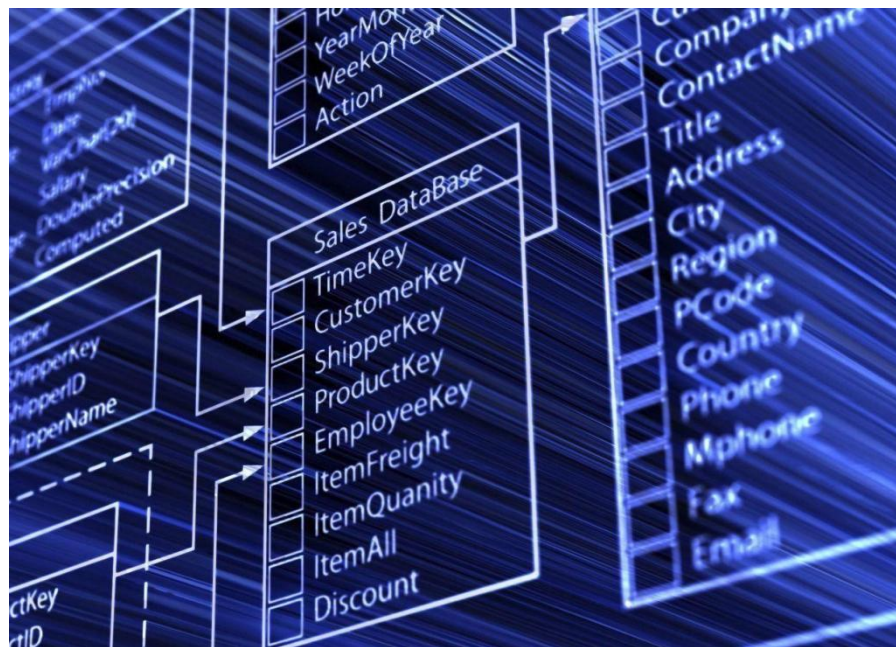


Fig 7.1 Data base

## 8- AJAX

### Ajax Tutorial

- AJAX tutorial covers concepts and examples of AJAX technology for beginners and professionals.
- AJAX is an acronym for Asynchronous JavaScript and XML. It is a group of inter-related technologies like JavaScript, DOM, XML, HTML/XHTML, CSS, XML Http Request etc.
- AJAX allows you to send and receive data asynchronously without reloading the web page. So, it is fast.
- AJAX allows you to send only important information to the server not the entire page. So only valuable data from the client side is routed to the server side. It makes your application interactive and faster.
- here are too many web applications running on the web that are using ajax technology like Gmail, Facebook, twitter, google map, YouTube etc.

### How AJAX Works?

- AJAX communicates with the server using XML Http Request object. Let's try to understand the flow of ajax or how ajax works by the image displayed below.
- As you can see in the above example, XML Http Request object plays an important role.
- User sends a request from the UI and a JavaScript call goes to XML Http Request object.
- HTTP Request is sent to the server by XML Http Request object.
- Server interacts with the database using JSP, PHP, Servlet, ASP.net etc.
- Data is retrieved.
- Server sends XML data or JSON data to the XML Http Request callback function.
- HTML and CSS data is displayed on the browser.

Servlet technology is used to create a web application (resides at server side and generates a dynamic web page).

- Servlet technology is robust and scalable because of java language. Before Servlet, CGI (Common Gateway Interface) scripting language was common as a server-side programming language. However, there were many disadvantages to this technology. We have discussed these disadvantages below.
- There are many interfaces and classes in the Servlet API such as Servlet, Generic Servlet, HTTP Servlet, Servlet Request, Servlet Response, etc. Servlet can be described in many ways, depending on the context.
- Servlet is a technology which is used to create a web application.
- Servlet is an API that provides many interfaces and classes including documentation.

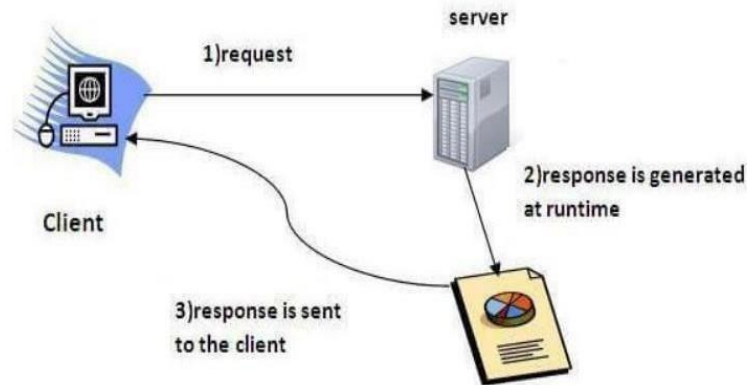


Fig 9.1- Server

Servlet is a class that extends the capabilities of the servers and responds to the incoming requests. It can respond to any requests.

- Servlet is a web component that is deployed on the server to create a dynamic web page.

- Hibernate is a Java framework that simplifies the development of Java application to interact with the database. It is an open source, lightweight, ORM (Object Relational Mapping) tool. Hibernate implements the specifications of JPA(Java Persistence API) for data persistence.

**ORM Tool:**

- An ORM tool simplifies the data creation, data manipulation and data access. It is a programming technique that maps the object to the data stored

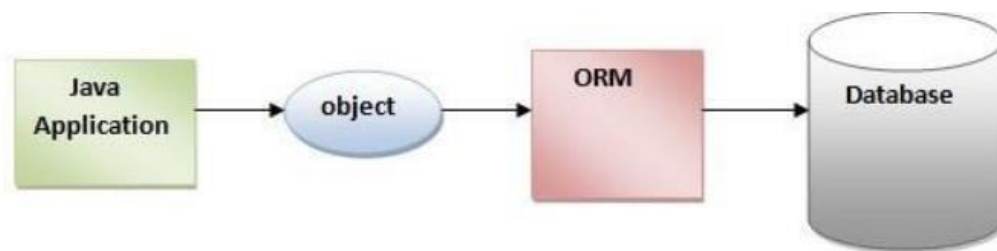


Fig 10.1 Orm tool

- The ORM tool internally uses the JDBC API to interact with the database.

**What is JPA?**

- Java Persistence API (JPA) is a Java specification that provides certain functionality and standard to ORM tools. The javax.persistence package contains the JPA classes and interfaces.

**Advantages of Hibernate Framework:**

- Following are the advantages of hibernate framework: Open Source and Light weight.
  - Hibernate framework is open source under the LGPL license and light weight. Fast Performance.
  - The performance of hibernate framework is fast because cache is internally used in hibernate framework. There are two types of cache in hibernate framework first level cache and second level cache. First level cache is enabled by default.
- Database Independent Query:

- HQL (Hibernate Query Language) is the object-oriented version of SQL. It generates the database independent queries. So, you don't need to write database specific queries. Before Hibernate, if database is changed for the project, we need to change the SQL query as well that leads to the maintenance problem. Automatic

Table Creation:

- Hibernate framework provides the facility to create the tables of the database automatically. So, there is no need to create tables in the database manually.

Simplifies Complex Join.

- Fetching data from multiple tables is easy in hibernate framework. Provides Query Statistics and Database Status.

- Hibernate supports Query cache and provide statistics about query and database status.

## 11- Spring Boot

Spring Boot Tutorial provides basic and advanced concepts of Spring

Our Spring Boot Tutorial is designed for beginners and professionals both.

Boot is a Spring module that provides the RAD

(Rapid Application Development) feature to the Spring framework. Our Spring Boot Tutorial includes all topics of Spring Boot such, as features, project, maven project, starter project wizard, Spring Initializer, CLI, applications, annotations, dependency management, properties, starters, Actuator, JPA, JDBC, etc.

- Spring Boot is a project that is built on the top of the Spring Framework. It provides an easier and faster way to set up, configure, and run both simple and web-based applications.
- It is a Spring module that provides the RAD (Rapid Application Development) feature to the Spring Framework. It is used to create a stand- alone Spring-based application that you can just run because it needs minimal Spring configuration.

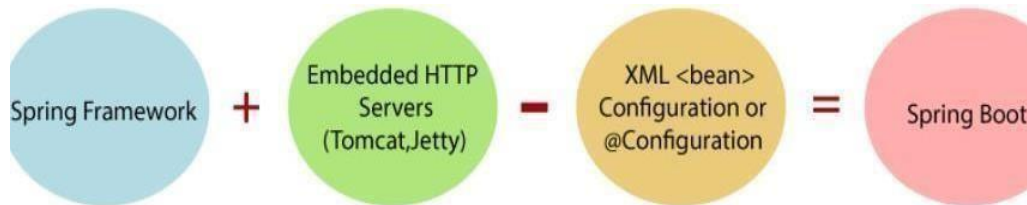


Fig 11.1- Spring boot

- In short, Spring Boot is the combination of Spring Framework and Embedded Servers.
- In Spring Boot, there is no requirement for XML configuration (deployment descriptor). It uses convention over configuration software design paradigm that means it decreases the effort of the developer.
- We can use Spring STS IDE or Spring Initializer to develop Spring Boot Java applications.



## ▪ **Why should we use Spring Boot Framework?**

- We should use Spring Boot Framework because:
  - The dependency injection approach is used in Spring Boot.
  - It contains powerful database transaction management capabilities.
- It simplifies integration with other Java frameworks like JPA/Hibernate ORM, Struts, etc.
- It reduces the cost and development time of the application.

Along with the Spring Boot Framework, many other Spring sister projects help to build applications addressing modern business needs. There are the following Spring sister projects are as follows:

- **Spring Data:** It simplifies data access from the relational and NoSQL databases.
- **Spring Batch:** It provides powerful batch processing.
- **Spring Security:** It is a security framework that provides robust security to applications.
- **Spring Social:** It supports integration with social networking like LinkedIn.
- **Spring Integration:** It is an implementation of Enterprise Integration Patterns. It facilitates integration with other enterprise applications using lightweight messaging and declarative adapters.

## **Conclusion:**

In conclusion, a Java Full Stack Developer requires a diverse range of technical and soft skills to develop and maintain web-based applications. They need to be proficient in both front-end and back-end development, understand security protocols, testing and debugging methodologies, design patterns, cloud technologies, and be customer-focused

## 13.CASE STUDY

### Introduction

LinkedIn is the world's largest professional networking site, connecting millions of professionals and businesses globally. With a vast user base and diverse functionality, LinkedIn requires a robust technology stack to manage real-time interactions, data processing, and user engagement. Java, known for its reliability and scalability, plays a crucial role in LinkedIn's backend development and service architecture.

### Background:

Founded in 2002, LinkedIn has evolved from a simple networking site to a comprehensive platform for job searching, professional development, and industry insights. As the platform has grown, so have its technical challenges, necessitating a reliable and efficient technology stack.

### Technology Stack:

#### 1. Backend Development:

**Java:** LinkedIn utilizes Java for developing backend services. Java's performance, scalability, and extensive ecosystem make it a fitting choice for handling high-volume user interactions and data processing.

**Frameworks:** Many backend services are built using the Spring Boot framework, which simplifies the development of microservices and RESTful APIs.

#### 2. Real-Time Data Processing:

**Apache Kafka:** Originally developed at LinkedIn, Kafka is a distributed event streaming platform used for building real-time data pipelines and streaming applications. It is implemented in Scala and Java and plays a pivotal role in facilitating communication between various services within LinkedIn.

#### 3. Frontend Development:

Although the frontend primarily utilizes JavaScript frameworks such as React.js, Java backend services provide essential RESTful APIs that deliver data to the frontend, ensuring a seamless user experience.

#### 4. Data Management:

LinkedIn employs a range of databases, including Oracle and MySQL for relational data management, and Voldemort, a Java-based distributed key-value store, for handling large datasets efficiently.

#### 5. Microservices Architecture:

LinkedIn's architecture is based on microservices, allowing teams to develop, deploy, and scale services independently. Many of these services are built using Java, ensuring that the system remains maintainable and scalable. Apache Hadoop is utilized for big data processing, incorporating various Java-based tools for analytics and data management.

## **Challenges and Solutions:**

### **1. Scalability:**

Challenge: Managing a rapidly growing user base and maintaining performance during peak usage.

Solution: By leveraging Java's capabilities and a microservices architecture, LinkedIn can scale services independently based on demand, ensuring reliable performance.

### **2. Real-Time Communication:**

Challenge: Delivering real-time updates to users (e.g., notifications, messages).

Solution: The integration of Apache Kafka allows LinkedIn to handle real-time data streams efficiently, providing users with instant notifications and updates.

### **3. Data Consistency:**

Challenge: Maintaining data consistency across various services and databases.

Solution: The use of transactional messaging with Kafka helps ensure that data remains consistent across different services, allowing for reliable updates and state management

## **Conclusion**

Java Full Stack development is a cornerstone of LinkedIn's technology infrastructure, enabling the platform to deliver a robust, scalable, and responsive experience to millions of users worldwide. The combination of Java with other technologies like Kafka and a microservices architecture allows LinkedIn to efficiently manage its vast array of services and functionalities. This case study highlights the significance of Java in creating high-performance applications that cater to the dynamic needs of users in a competitive professional networking landscape

